# Zero Trust Backplane — Agent Authentication Backlog

## What is authentication?

Authentication answers one narrow question: is this party actually who it claims to be? It is a distinct problem from authorization, which answers a different question: given a proven identity, what is it allowed to do? The two are sequential layers, not interchangeable — a system can enforce policy correctly while never once verifying who it was actually enforcing that policy against.

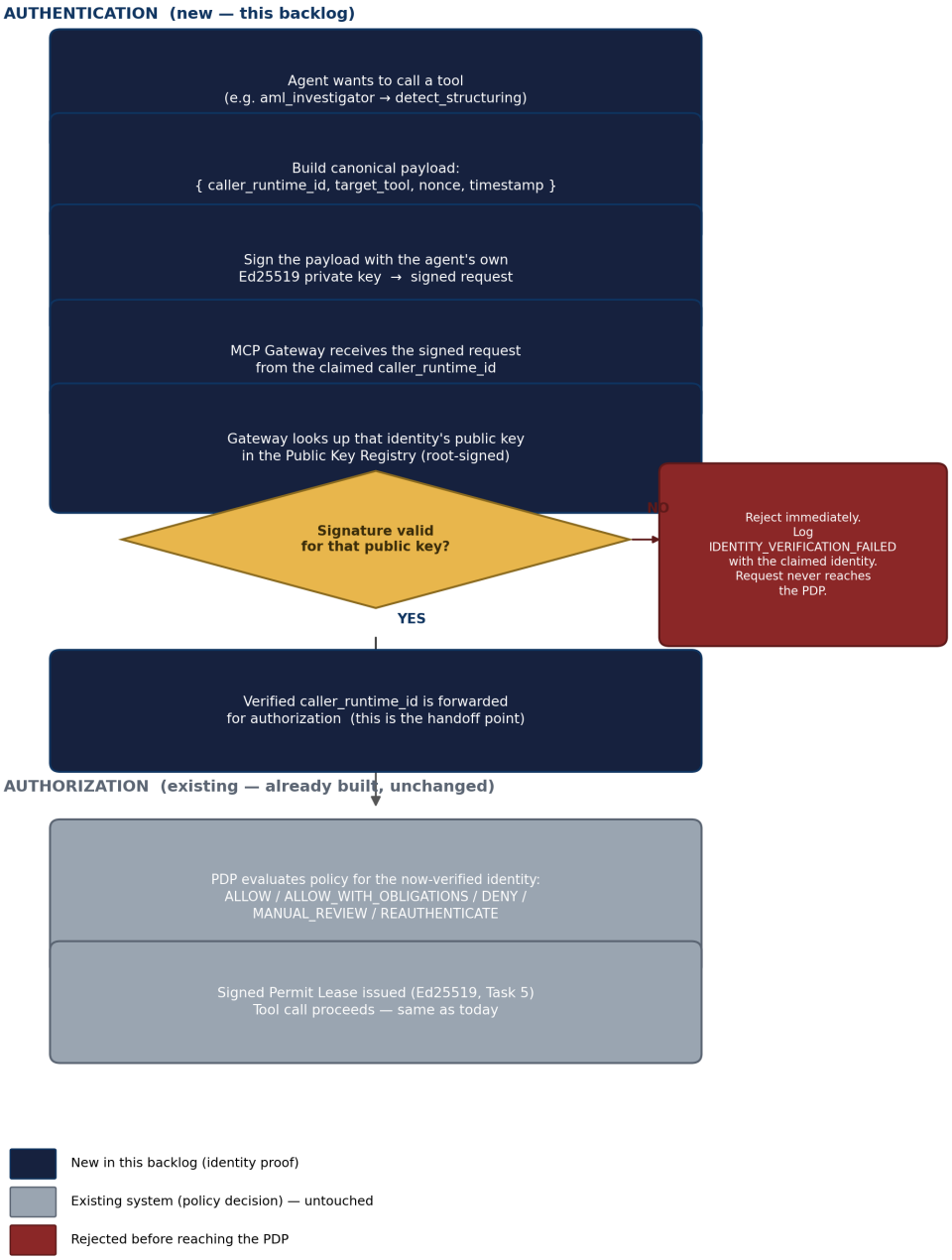
## How does it differ from what we have today?

Today, the Zero Trust Backplane only implements the second half — authorization. Every PDP decision (ALLOW, ALLOW_WITH_OBLIGATIONS, DENY, MANUAL_REVIEW, REAUTHENTICATE) is signed with Ed25519 via Task3PEPAdapter, so the decision itself cannot be forged after the fact. But the caller_runtime_id feeding into that decision — the identity claim — is just a string asserted by whoever is calling. Nothing today checks that a request claiming to be aml_investigator actually originated from that agent. In short: the policy gate is real and cryptographically enforced, but it never checks ID at the door before applying that policy.

## Which aspect are we implementing?

This backlog implements impersonation defense specifically: proving a caller is who it claims to be, using the same Ed25519 primitive already trusted elsewhere in this system. It deliberately does not attempt to solve a harder, separate problem — an authenticated agent whose own reasoning has been manipulated (prompt injection, a poisoned tool result, a jailbreak) is still a legitimate, correctly-signed caller from the gateway's point of view. Authentication proves WHO is calling; it says nothing about whether that caller's current behavior can be trusted. That second problem is real and worth tracking, but it is out of scope here — this phase closes the identity-spoofing gap on its own, bounded terms.

## Authentication flow




*Where authentication sits relative to the existing system. Every navy step below is new in this backlog; every gray step already exists and is unchanged — authentication only decides whether a request is even allowed to reach that existing PDP flow.*

## Implementation approach

This backlog closes the gap above by giving every agent identity its own Ed25519 keypair — the same primitive already used for Permit Lease signing — and requiring a verified signature before the PDP is even consulted. All five tasks live inside Citi_P3_Zero_Trust_Backplane/, in a new zero_trust_task6_agent_identity/ module, consistent with how Tasks 1-5 are already organized. The one necessary exception is a thin signing-client shim imported by citi_banking_adk so agents can sign their own outgoing requests — kept intentionally minimal so the Backplane remains the source of truth for identity, not the application code.

### Build order

Build order matters: Tasks 1 and 2 (key generation + registry) are the foundation — nothing else is meaningful without them. Task 3 (signed requests + verification) is where authentication actually starts being enforced; Tasks 1/2 alone only generate and store keys, they don't check anything yet. Tasks 4 and 5 (rotation, revocation) are lifecycle management built on top of a working Task 3 — they cannot be started first.

## Known failure patterns to guard against — read before starting

This project's Zero Trust debugging history already surfaced two real, previously-undetected bugs of the exact shape this backlog is likely to reproduce if these lessons aren't applied deliberately. Both are folded into the specific tasks below, but are worth stating plainly up front:

**A discovery/parsing step can silently return an empty or wrong result without crashing.**

Task 2's scanner (extract_string_list) checked isinstance(node, ast.List), but the value it received had already been resolved to a plain Python list by an earlier call — the check never matched, and every agent's tools/sub_agents silently came back as []. It ran without error for a long time before being caught. Applies directly to AUTHN-1, which derives agent identities from the same scanner output.

**A caller can invoke methods on an object that were never actually defined on it, undetected, if that code path is never exercised end-to-end.**

native_mas_runtime/engine.py called _check_pre_obligations and _apply_post_obligations on Task3PEPAdapter — methods that class never had — because no tool call had ever previously reached the ALLOW_WITH_OBLIGATIONS path in a real run. Applies directly to AUTHN-3, which wires new verification code into that same engine.py from the other side.

**A hardcoded whitelist/allowlist not updated for a new feature makes correct data silently invisible, not obviously broken.**

runner.py's IMPORTANT_RESULT_KEYS never had the AML output keys added, so a fully working investigation produced no visible result in the compact trace for the whole time AML was being tested. Applies to AUTHN-3's new audit event types (IDENTITY_VERIFICATION_FAILED, IDENTITY_REVOKED) — check every existing place event_type values are enumerated, not just the one obvious integration point.

**Silent fallback on unexpected input hides real failures instead of surfacing them.**

A truncated/malformed LLM JSON response was silently treated as {} by _extract_json, causing a genuinely high-risk case to skip SAR escalation with no warning at all. Applies to every task here: a verification failure, a registry lookup miss, a revoked key must always produce a visible, attributable audit entry — never a quiet skip.

# AUTHN-1 — Per-Agent Ed25519 Key Generation & Bootstrap

### User Story

*As a Zero Trust engineer, I want every agent identity to have its own Ed25519 keypair generated at setup time, so that each agent can later prove its identity cryptographically instead of the pipeline trusting an asserted runtime_id string.*

### To-do List

- Add a keygen module reusing cryptography.hazmat.primitives.asymmetric.ed25519 (already a dependency via zero_trust_task5's Task3PEPAdapter)
- Derive the list of agent identities to bootstrap from Task 2's scanner output (agents_draft.yaml) rather than hand-maintaining a list, so newly discovered agents automatically get key entries on the next --zt-refresh
- Known-failure-pattern check: do not trust that the scanner output is correct just because bootstrap runs without error — see 'Known failure patterns' above (extract_string_list silently returned [] for a long time). Explicitly assert the derived identity list is non-empty and contains every expected agent name before generating keys from it
- Store private key material outside of anything tracked/committed (a gitignored keys/ directory, the same pattern already used to exclude citi_banking_adk/.env)
- Provide a CLI entrypoint (bootstrap_agent_keys.py) run once per environment setup to generate the full keyset
- Fail loudly (not silently skip) if bootstrap is run for an agent that already has a key, unless --force is explicitly passed

### Deliverables

- zero_trust_task6_agent_identity/src/zt_task6/keygen.py
- zero_trust_task6_agent_identity/bootstrap_agent_keys.py
- .gitignore entry for the private-key directory

### Directory

`Citi_P3_Zero_Trust_Backplane/zero_trust_task6_agent_identity/`

### Tests to Validate Correctness

- The identity list derived from Task 2's scanner output is non-empty and includes every real agent name (aml_investigator, sar_drafting_agent, front/mid/back office agents, etc.) — not just 'the function returns without raising'
- Running bootstrap twice does not silently overwrite an existing agent's key without --force
- Each generated private key is a valid Ed25519 key (round-trip sign/verify test)
- Generated private key files are not world-readable (file permission check, e.g. mode 0600)

# AUTHN-2 — Public Key Registry (Trust Anchor)

### User Story

*As the MCP gateway, I want a single trusted place to look up which public key belongs to which agent identity, so that I can verify a signature without trusting the caller's own claim about which key it used.*

### To-do List

- Design a registry format mapping agent_identity -> public_key (public keys only — the registry must never contain private key material)
- Populate the registry from Task 1's keygen output as part of the bootstrap flow
- Protect the registry's own integrity: sign the registry file itself with the Backplane's root key, so a tampered registry entry is detectable rather than silently trusted — otherwise this just relocates the trust problem to an unprotected file
- Add a registry lookup function used by Task 3's verification step — and use that exact same function in this task's own tests, rather than a separate test-only stand-in that could silently drift from what Task 3 actually calls
- Define fail-closed behavior: an unknown agent identity returns no key, never a default/fallback key

### Deliverables

- zero_trust_task6_agent_identity/src/zt_task6/registry.py
- registry schema + loader (YAML or SQLite, consistent with existing aml_alerts table style)
- Root-signing mechanism for the registry file

### Directory

`Citi_P3_Zero_Trust_Backplane/zero_trust_task6_agent_identity/`

### Tests to Validate Correctness

- Lookup returns the correct public key for a known agent identity
- Lookup fails closed (raises / returns None, not a default-allow) for an unknown identity
- A tampered registry file (modified public key, invalid root signature) is detected and rejected before any lookup succeeds
- Registry reload picks up newly bootstrapped agent keys without a full redeploy

# AUTHN-3 — Signed Requests + Gateway-Side Verification

### User Story

*As the MCP gateway, I want to verify a cryptographic signature on every tool-call request before asking the PDP for an authorization decision, so that the caller_runtime_id fed into policy is proven, not merely asserted.*

### To-do List

- Define the canonical signed payload format — {caller_runtime_id, target_tool, nonce, timestamp} — ONCE, in a single shared module that both signing.py and verification.py import. Never let each side independently reimplement the format: any drift between them means signatures silently fail every time, with no obvious cause
- Add replay protection: reject signatures outside a short validity window, and reject a nonce reused within that window
- Add a lightweight signing client, imported by citi_banking_adk, so agents can sign outgoing tool-call requests with their own private key from Task 1 — kept minimal, this is the one necessary cross-repo touchpoint
- Known-failure-pattern check before writing verification.py's integration: grep native_mas_runtime/engine.py for the exact point TOOL_AUTHORIZATION events are generated and match the REAL call signature it expects — do not assume the interface (see 'Known failure patterns' above: engine.py once called two methods on Task3PEPAdapter that were never defined, undetected because that path had never actually been exercised end-to-end)
- Wire verification into native_mas_runtime/engine.py at that point — verification must pass before find_tool_placement()/begin_tool() proceed to the PDP
- On verification failure, emit a distinct audit event type (IDENTITY_VERIFICATION_FAILED) rather than falling through to the existing DENY path, so a bad identity and a bad policy decision stay distinguishable in the audit log
- Known-failure-pattern check: grep every existing place event_type values are enumerated or whitelisted (e.g. runner.py's IMPORTANT_RESULT_KEYS-style lists, any audit-log summarizer/formatter) and add the new event types there too — otherwise a working verification layer can produce correct events that are silently invisible in every existing view, exactly like AML's result keys were for a full session before being caught

### Deliverables

- zero_trust_task6_agent_identity/src/zt_task6/signing.py (agent-side)
- zero_trust_task6_agent_identity/src/zt_task6/verification.py (gateway-side)
- zero_trust_task6_agent_identity/src/zt_task6/payload_format.py (single shared definition of the canonical signed payload, imported by both signing and verification)
- Minimal signing-client shim imported by citi_banking_adk/bank_adk/
- Updated native_mas_runtime/engine.py integration

### Directory

`Citi_P3_Zero_Trust_Backplane/zero_trust_task6_agent_identity/ (+ a thin import in citi_banking_adk/bank_adk/)`

### Tests to Validate Correctness

- A correctly signed request from a real agent identity passes verification
- A request claiming an identity it doesn't hold the private key for is rejected

- A replayed (previously-used) signed request is rejected
- An expired-timestamp signed request is rejected
- End-to-end test runs through the REAL python run_cli.py --zero-trust CLI path, not a mocked unit test — neither of this project's two real integration bugs would have been caught by isolated unit tests of the individual modules. Confirm a genuine CUST-STRUCT structuring investigation still succeeds with verification enabled, and its audit log shows a distinct identity-verification event ahead of the existing TOOL_AUTHORIZATION entry

# AUTHN-4 — Key Rotation

### User Story

*As a Zero Trust engineer, I want to rotate an agent's keypair on a schedule or on demand, so that a key's exposure window is bounded even without a known compromise.*

### To-do List

- Define a rotation policy (e.g. max key age) and procedure: generate a new keypair, publish the new public key to the registry alongside the old one for a grace period, retire the old key after the grace period
- Ensure in-flight requests signed with the old-but-not-yet-retired key still verify during the grace period, so a hard cutover doesn't break active agents
- Add a rotate_agent_key CLI command
- Every rotation updates the registry's root signature, so the change itself stays integrity-protected (Task 2's guarantee must hold after every rotation, not just at initial bootstrap)

### Deliverables

- zero_trust_task6_agent_identity/src/zt_task6/rotation.py
- rotate_agent_key.py CLI entrypoint
- Grace-period handling in registry.py (multiple valid keys per identity, each with its own validity window)

### Directory

`Citi_P3_Zero_Trust_Backplane/zero_trust_task6_agent_identity/`

### Tests to Validate Correctness

- A request signed with the newly rotated key verifies successfully immediately after rotation
- A request signed with the old key still verifies during the grace period
- A request signed with the old key is rejected once the grace period expires
- Rotation updates the registry's root signature; a rotation event that skips re-signing the registry is itself detected as tampering

# AUTHN-5 — Key Revocation

### User Story

*As a Zero Trust engineer, I want to immediately invalidate a specific agent's key if it's suspected compromised, so that a leaked key can't be used to impersonate that agent while a rotation is pending.*

### To-do List

- Add a revocation status per key in the registry (active / revoked), checked during verification
- Revocation takes effect immediately — no grace period, unlike rotation, since this is the active-incident path, not routine hygiene
- Add a revoke_agent_key CLI command that logs a distinct audit event (e.g. IDENTITY_REVOKED) recording who revoked what and when
- Document and enforce the fail-closed fallback: an agent whose only key is revoked cannot authenticate anywhere in the pipeline until a new key is issued and registered — no automatic fallback to an unauthenticated path

### Deliverables

- zero_trust_task6_agent_identity/src/zt_task6/revocation.py
- revoke_agent_key.py CLI entrypoint
- Revocation check added to verification.py's lookup path

### Directory

`Citi_P3_Zero_Trust_Backplane/zero_trust_task6_agent_identity/`

### Tests to Validate Correctness

- A request signed with a revoked key is rejected even though the signature itself is cryptographically valid
- Revocation takes effect immediately — a request one second after revocation still fails
- An agent whose only key is revoked cannot authenticate anywhere in the pipeline until a new key is bootstrapped and registered
- The audit log records a distinct, attributable IDENTITY_REVOKED event